AMR Extension of LaRT: Octree Structure and Raytrace Algorithm

Contents

1. Overview **2**

2. AMR Grid Data Structure **3**

2.1 Octree Topology . . . 3

2.2 Refinement Constraints: Unrestricted Octree . . . 3

2.3 Arrays Stored per Cell . . . 3

2.4 Coordinate Convention . . . 4

3. Building the Octree **5**

3.1 Tree Construction: `amr_build_tree` . . . 5

3.2 Face-Neighbor Precomputation: `amr_build_neighbors` . . . 5

4. Cell-Crossing Algorithms **6**

4.1 Finding a Leaf: `amr_find_leaf` . . . 6

4.2 Cell-Exit Distance: `amr_cell_exit` . . . 6

4.3 Next Leaf After a Face Crossing: `amr_next_leaf` . . . 6

5. Grid Initialization **7**

5.1 Input Data Format . . . 7

5.2 Biconical Geometry Mask . . . 7

5.3 Tau Normalization by Pole Traversal . . . 8

6. Photon Propagation **8**

6.1 Main Raytrace Routine: `raytrace_to_tau_amr` . . . 8

6.2 Frequency Shift at Cell Boundaries . . . 9

6.3 Overshoot Correction . . . 9

6.4 Photon Escape and Spectrum Accumulation . . . 10

6.5 Boundary Handling: Periodicity and Symmetry Planes . . . 10

6.6 Other Raytrace Routines . . . 11

7. Scattering **11**

7.1 Resonance Scattering . . . 11

7.2 Dust Scattering . . . 13

7.3 Molecular-Hydrogen (H_2) Absorption on the AMR Grid . . . 13

7.4 Multi-Level Resonance Transitions . . . 13

8. Output Normalization 13

9. Core-Skip Acceleration 14

10. HEALPix Inside-Observer Mode 14

11. Validation 16

12. Summary of Key Design Choices 17

1. Overview

LaRT (Lyman-alpha Radiative Transfer) is a Monte Carlo radiative transfer code for Lyman-α photon propagation in astrophysical media. The base code (`LaRT_combined`) uses a uniform Cartesian grid. `LaRT_v2.00` extends it with an Adaptive Mesh Refinement (AMR) octree grid, enabling direct use of outputs from cosmological simulation codes such as RAMSES.

AMR mode is activated with `par%grid_type = 'amr'` in the input namelist (the legacy flag `par%use_amr_grid = .true.` is still accepted as a deprecated alias). Grid-type selection uses a Fortran 2003 class hierarchy defined in `grid_class.f90`: an abstract base type `grid_base_type` provides deferred type-bound procedures for all physics operations, and the concrete types `grid_car_type` and `grid_amr_type` implement them for Cartesian and AMR grids, respectively. The outer simulation loop (`run_simulation_mod.f90`) dispatches all calls through the polymorphic `grid` variable, and photon generation is handled by the module-specific routines `gen_photon_car` (module `photon_mod_car`) and `gen_photon_amr` (module `photon_mod_amr`).

The new source files are:

File	Role
`octree_mod.f90`	Octree data type, tree operations, cell-crossing routines
`grid_mod_amr.f90`	Grid creation, opacity normalization, frequency grid
`raytrace_amr.f90`	Photon propagation through AMR cells
`scattering_amr.f90`	Resonance and dust scattering (cell-by-cell parameters)
`peelingoff_amr.f90`	Peel-off observer calculation
`read_generic_amr.f90`	Reader for generic AMR data (text, FITS, HDF5) with optional physics columns
`physics_amr_mod.f90`	Shared physics functions: CIE neutral fraction, Laursen+09 dust, Case B emissivity
`gen_photon_amr.f90`	Photon generation for AMR grid (module `photon_mod_amr`)
`gen_photon_car.f90`	Photon generation for Cartesian grid (module `photon_mod_car`)
`grid_class.f90`	OOP class hierarchy: abstract `grid_base_type`, concrete `grid_car_type` and `grid_amr_type`

2. AMR Grid Data Structure

2.1 Octree Topology

The AMR grid is represented as a *linear octree*: a flat Fortran array of cells indexed $1, 2, \ldots, N_{\text{cells}}$. Cell 1 is the root, which covers the entire computational box. Each internal cell has exactly eight children obtained by bisecting the parent along all three coordinate axes.

The child octant index is encoded as

$$i_{\text{oct}} = 1 + i_x + 2\,i_y + 4\,i_z, \tag{1}$$

where $i_x = 1$ if $x \geq c_x$ (the cell center), otherwise $i_x = 0$, and similarly for i_y and i_z. Hence octant indices run from 1 to 8.

2.2 Refinement Constraints: Unrestricted Octree

Unlike RAMSES (and RASCAS, which reuses the RAMSES grid structure), which enforce a so-called *2:1 balance* or *graded refinement* constraint requiring that the refinement levels of any two face-adjacent leaf cells differ by at most one, the LaRT generic AMR octree is *unrestricted*: any two adjacent leaves may differ by an arbitrary number of levels. Neither the file reader (`read_generic_amr.f90`) nor the tree builder (`amr_build_tree`) imposes or checks the 2:1 condition; leaf cells are inserted at whatever level the input data specify.

Support for arbitrary level jumps is built into the traversal layer:

- The face-neighbor table (Section 3.2) returns the face neighbor on the same level when it is itself a leaf, a *coarser* leaf when refinement is absent on the other side, or an *internal* cell when the neighbor side is more refined.
- `amr_next_leaf` (Section 4.3) handles the more-refined case by descending the octree from the stored internal cell through as many levels as needed until a leaf is reached. The descent cost is $O(\Delta\ell)$ with $\Delta\ell = \ell_{\max} - \ell_i$, so any level difference is accommodated at no algorithmic penalty beyond the descent depth.

RAMSES snapshots arriving through `convert_ramses_to_generic` are of course already 2:1 balanced and remain so in the generic file; data from sources that do not impose the constraint (manually constructed grids, the bundled Python generators such as `make_amr_sphere_radial.py`, custom converters) are accepted as-is.

2.3 Arrays Stored per Cell

The Fortran derived type `amr_grid_type` (defined in `octree_mod.f90`) stores the following arrays of size N_{cells} for the tree topology:

Array	Dimension	Content
`parent(:)`	$N_{\rm cells}$	parent cell index (0 for root)
`children(:,:)`	$8 \times N_{\rm cells}$	child indices; 0 = leaf
`level(:)`	$N_{\rm cells}$	refinement level (0 = root)
`cx, cy, cz`	$N_{\rm cells}$	cell center coordinates (code units)
`ch(:)`	$N_{\rm cells}$	cell half-width (code units)
`ileaf(:)`	$N_{\rm cells}$	> 0: leaf index; 0: internal
`icell_of_leaf(:)`	$N_{\rm leaf}$	cell index for each leaf
`neighbor(:,:)`	$6 \times N_{\rm cells}$	face-neighbor table (see Section 3.2)

Leaf cells (`ileaf`$(i) > 0$) carry physical data stored in separate arrays of size $N_{\rm leaf}$:

Array	Symbol	Physical meaning
`rhokap`	$\bar{\kappa}_0$	HI line-center opacity per code-unit length
`rhokapD`	$\bar{\kappa}_d$	dust opacity per code-unit length
`Dfreq`	$\Delta\nu_D$	local Doppler frequency [Hz]
`voigt_a`	a	Voigt damping parameter $a = A_{21}/(4\pi\Delta\nu_D)$
`vfx, vfy, vfz`	$\tilde{v}_x, \tilde{v}_y, \tilde{v}_z$	bulk velocity / local $v_{\rm th}$

The spectral output arrays (`Jout`, `Jin`, `Jabs`) have size N_ν (the number of frequency bins) and are also members of `amr_grid_type`. All photon-weight accumulation in AMR mode goes into `amr%J*` directly: `Jout` and `Jmu` from `raytrace_amr.f90`, `Jabs` from `scattering_amr.f90`, and `Jin` from `generate_photon_amr.f90`. At reduction time `output_reduce_amr` (or `output_reduce_inside_amr` on the HEALPix path) calls `MPI_ALLREDUCE` on each `amr%J*` and copies the result into the corresponding `g%J*` alias so that the shared write-output routines see populated arrays. (Note: in v2.00 the `Jin` accumulation site lives in the shared `generate_photon.f90`; that path was previously writing to `grid%Jin` unconditionally and was fixed in 2026-04-30 to dispatch on `par%use_amr_grid`. v2.10 was unaffected because its `generate_photon_amr.f90` already writes `amr%Jin` natively.)

2.4 Coordinate Convention

All cell positions and path lengths are stored in *code units* (kpc, pc, or whatever unit the input data use). The conversion factor `par%distance2cm` is used once when computing opacities:

$$\bar{\kappa}_0\ [\text{code-unit}^{-1}] = n_{\rm HI}\,[\text{cm}^{-3}] \times \frac{\sigma_0}{\Delta\nu_D} \times d_{\rm cm}, \tag{2}$$

where $d_{\rm cm}$ is the number of centimeters per code unit. After that, all quantities are dimensionless with respect to code units, exactly mirroring the Cartesian convention.

The box is always **centered at the origin**: all leaf coordinates lie in $[-L_{\rm box}/2, +L_{\rm box}/2]$ on each axis, and the octree root spans the same range. This is required because the peel-off geometry and the default point-source position (`xs_point = ys_point = zs_point = 0`) both assume $(0,0,0)$ to be the box center. Input readers are responsible for mapping raw data to this centered convention before passing coordinates to `amr_build_tree`.

3. Building the Octree

3.1 Tree Construction: `amr_build_tree`

`amr_build_tree` takes flat arrays of leaf-cell center coordinates (`xleaf`, `yleaf`, `zleaf`), AMR levels (`leaf_level`), and box bounds, and constructs the octree by inserting each leaf cell top-down from the root.
The algorithm is as follows:

1. Initialize the root cell (index 1) with half-width $h = L_{\mathrm{box}}/2$.
2. For each leaf $\ell = 1, \ldots, N_{\mathrm{leaf}}$:
   (a) Start at the root (`icell` = 1).
   (b) For each refinement level from 0 to $\ell_{\mathrm{leaf}} - 1$:
      i. Compute the octant index i_{oct} using the leaf center.
      ii. If `children(`i_{oct}`, icell)` is zero, create a new internal node with half-width $h/2$ and center offset $\pm h/2$ from the parent center.
      iii. Descend: `icell` $\leftarrow$ `children(`i_{oct}`,icell)`.
   (c) Mark the reached cell as leaf ℓ: `ileaf(icell)` = ℓ.

The array is grown dynamically (doubled) if the pre-allocated size is exceeded.

3.2 Face-Neighbor Precomputation: `amr_build_neighbors`

After the tree is built, `amr_build_neighbors` pre-computes a $6 \times N_{\mathrm{cells}}$ neighbor table. Entry `neighbor`(f, i) stores the cell index of the neighbor of cell i across face f, using the face convention:

$$f = 1{:}+x, \quad 2{:}-x, \quad 3{:}+y, \quad 4{:}-y, \quad 5{:}+z, \quad 6{:}-z. \tag{3}$$

A value of 0 means the face is on the domain boundary.
For each cell i, the routine probes the point just outside each face (offset $h \times 10^{-8}$ from the face) and descends the tree to the cell at level $\leq \ell_i$ that contains that probe point:

$$\texttt{neighbor}(f, i) = \texttt{amr_find_cell_at_level}\left(c_i + h_f \hat{e}_f,\ \ell_i\right), \tag{4}$$

where h_f is the signed probe offset along face direction f and $\hat{e}_f$ is the corresponding unit vector.
The result may be a leaf cell at the same level, a leaf at a coarser level (if the grid is not refined there), or an internal cell at the same level (if the neighbor side is *more* refined than cell i). The last case is handled at runtime by `amr_next_leaf` (Section 4.3).
This table is built once and then used for O(1) boundary crossings throughout the simulation.

4. Cell-Crossing Algorithms

4.1 Finding a Leaf: `amr_find_leaf`

`amr_find_leaf`(x,y,z) descends the octree from the root, choosing the correct child octant at each level using the octant formula (Equation 1), until it reaches a cell with `ileaf` > 0. The result is the leaf index $\ell \in \{1,\ldots,N_{\text{leaf}}\}$. The depth is at most $\ell_{\max}$, the maximum AMR level.

This function is used only at photon injection (once per photon) and as a fallback if `photon%icell_amr` is not yet set.

4.2 Cell-Exit Distance: `amr_cell_exit`

Given a photon at position (x,y,z) inside cell i with direction (k_x,k_y,k_z), `amr_cell_exit` computes the distance t_{exit} to the nearest cell face and identifies which face f is crossed.

For each axis $\alpha \in \{x,y,z\}$ with cell center c_α and half-width h, the distances to the $\pm$ faces are:

$$t_\alpha^+ = \frac{c_\alpha + h - \alpha}{k_\alpha}, \quad t_\alpha^- = \frac{c_\alpha - h - \alpha}{k_\alpha}, \tag{5}$$

with $t = \infty$ (implemented as `hugest`) when the direction component is zero or points away from the face. The exit is the minimum over all six half-space times:

$$t_{\text{exit}} = \min(t_x^+, t_x^-, t_y^+, t_y^-, t_z^+, t_z^-), \tag{6}$$

and f is the index of the minimizing element (1–6 in the convention above).

4.3 Next Leaf After a Face Crossing: `amr_next_leaf`

After the photon crosses face f of cell i and the position has been updated to (x,y,z) at the face, `amr_next_leaf`(i,f,x,y,z) returns the leaf index of the cell the photon enters. The algorithm is:

1. Look up `ineigh` = `neighbor`(f,i). If zero, the photon has left the domain; return 0.
2. If `ileaf(ineigh)` > 0, the neighbor is already a leaf; return `ileaf(ineigh)`.
3. Otherwise the neighbor is an internal cell at a finer level. Descend:
   (a) Compute i_{oct} from the actual crossing position (x,y,z) and the center of `ineigh`.
   (b) `ineigh` $\leftarrow$ `children`$(i_{\text{oct}}$, `ineigh`$)$.
   (c) Repeat until `ileaf(ineigh)` > 0.
4. Return `ileaf(ineigh)`.

Step 1 is O(1); the descent in step 3 traverses at most $\Delta\ell = \ell_{\max} - \ell_i$ extra levels, which is typically one or two. In practice this makes cell-boundary crossings effectively O(1) for any AMR grid.

5. Grid Initialization

`grid_create_amr` in `grid_mod_amr.f90` performs the following sequence:

1. **Read leaf-cell data.** MPI rank 0 reads the generic AMR file (text, FITS, or HDF5) via `read_generic_amr.f90` and broadcasts all arrays—including any optional physics columns—to the other ranks. Each rank therefore holds a full copy of the tree (appropriate for moderate grid sizes).

2. **Build octree.** Call `amr_build_tree` and `amr_build_neighbors`.

3. **Compute cell-by-cell Doppler and Voigt parameters.** For leaf ℓ:

$$\Delta\nu_{D,\ell} = \frac{b_{D,\ell}}{\lambda_0}, \quad b_{D,\ell} = \sqrt{\left(v_{\mathrm{th},1}\sqrt{T_\ell}\right)^2 + b_{\mathrm{turb}}^2}, \quad a_\ell = \frac{A_{21}}{4\pi\,\Delta\nu_{D,\ell}}, \tag{7}$$

   where $v_{\mathrm{th},1} = \sqrt{2k_B/m_H}$ is the thermal velocity per $\sqrt{T}$ and $b_{\mathrm{turb}} =$ `par%bturb` is a spatially uniform turbulent velocity added in quadrature. The thermal width uses the *cell-by-cell* data-file temperature T_ℓ; `par%bturb` ≤ 0 recovers the pure thermal width.

4. **Compute cell-by-cell opacity and velocity.**

$$\bar{\kappa}_{0,\ell} = n_{\mathrm{HI},\ell}\,\frac{\sigma_0}{\Delta\nu_{D,\ell}}\,d_{\mathrm{cm}}, \tag{8}$$

$$\tilde{v}_{\alpha,\ell} = \frac{v_{\alpha,\ell}[\mathrm{km\,s^{-1}}]}{b_{D,\ell}[\mathrm{km\,s^{-1}}]}, \tag{9}$$

   where σ_0 is the hydrogen Lyman-α cross-section at line center and d_{cm} is centimeters per code unit.

5. **Normalize opacity to `taumax`.** See Section 5.3.

6. **Set up frequency grid and output arrays.**

5.1 Input Data Format

LaRT reads only the *generic AMR format* via the module `read_generic_amr.f90`. RAMSES binary data must be converted to this format using the standalone converter before running LaRT (see the user manual).

The generic format supports text (`.dat`), FITS (`.fits`, `.fits.gz`), and HDF5 (`.h5`, `.hdf5`). Nine columns are mandatory (x, y, z, level, nH, T, vx, vy, vz); six optional columns (`metallicity`, `xHI`, `n_e`, `n_ion`, `emissivity`, `ndust`) are detected by name in HDF5/FITS files and used directly when present.

The code unit is set by `par%distance_unit` (e.g., `'kpc'`). Leaf coordinates must be centered at the origin, i.e., in $[-L_{\mathrm{box}}/2, +L_{\mathrm{box}}/2]$ on each axis. The Python `AMRGrid` class writes coordinates in this convention.

5.2 Biconical Geometry Mask

When the input parameter `par%cone_opening` > 0 (and < 90), a biconical geometry mask is applied after the cell-by-cell opacity is set up and before the optical-depth normalization. For every leaf cell ℓ, if $|\cos\theta_\ell| <$ cos(`cone_opening`) (where θ_ℓ is the polar angle of the cell center measured from the z-axis), both the line opacity $\rho\kappa_\ell$ and the dust opacity

$\rho\kappa_{d,\ell}$ are set to zero. This confines the scattering medium to two opposing cones along the z-axis with half-opening angle $\alpha_{\rm cone}$.

The standalone AMR grid generator (`make_amr_sphere_radial.x`) also supports `--cone_opening`: leaves outside the cone are written with zero density and zero velocity, and cells intersecting the cone boundary surface are refined to `boundary_level_max` when `--refine_boundary` is active.

5.3 Tau Normalization by Pole Traversal

The optical depth normalization convention in `LaRT_v2.00` matches the Cartesian version: `taumax` is the line-center optical depth from the box center to the $+z$ boundary along the z-axis.

After the raw opacities are set, a ray is traced from the box center in the $+z$ direction using the same AMR raytrace kernel:

$$\tau_{\rm pole} = \sum_{\ell} \bar{\kappa}_{0,\ell}\, \phi(0, a_\ell)\, \Delta s_\ell, \tag{10}$$

where $\phi(x,a)$ is the Voigt profile at $x = 0$ and Δs_ℓ is the path-length through leaf ℓ along the ray. The normalization factor is

$$f_{\rm norm} = \frac{\texttt{taumax}}{\tau_{\rm pole}}, \tag{11}$$

and all opacities are scaled: $\bar{\kappa}_{0,\ell} \leftarrow f_{\rm norm}\, \bar{\kappa}_{0,\ell}$. The same factor is applied to dust opacity if present.

This pole-traversal approach is exact for any opacity distribution, in contrast to the volume-average approximation $\tau \approx \langle \bar{\kappa}_0 \rangle\, L_{\rm box}$ that underestimates $\tau_{\rm pole}$ for centrally concentrated clouds such as a sphere.

6. Photon Propagation

6.1 Main Raytrace Routine: `raytrace_to_tau_amr`

This routine propagates a photon through the AMR grid until it accumulates a target optical depth $\tau_{\rm in}$ (sampled as $-\ln\xi$ for a uniform random ξ) or leaves the box.

The photon state is represented by:

- (x, y, z) — current absolute position in code units,
- (k_x, k_y, k_z) — unit direction vector,
- x_ν — frequency offset in units of the local Doppler width (comoving frame of the current cell),
- ℓ — current leaf index (`photon%icell_amr`).

The loop at each step n:

1. Convert leaf index to cell index: i = `icell_of_leaf`(ℓ).
2. Compute the exit distance and exit face: $(t_{\rm exit}, f)$ = `amr_cell_exit`$(i, x, y, z, k_x, k_y, k_z)$.

3. Evaluate the opacity at the current frequency:

$$\kappa = \bar{\kappa}_{0,\ell}\,\phi(x_\nu, a_\ell) + \bar{\kappa}_{d,\ell} \quad \text{(with dust, if present)}, \tag{12}$$

   where $\phi(x,a)$ is the Voigt function.

4. Update accumulated optical depth: $\tau \leftarrow \tau + \kappa\, t_{\rm exit}$.

5. **If** $\tau \geq \tau_{\rm in}$: backtrack to the exact interaction point (Section 6.3) and exit the loop.

6. Otherwise: advance position to the face,

$$(x, y, z) \leftarrow (x + t_{\rm exit}\,k_x,\ y + t_{\rm exit}\,k_y,\ z + t_{\rm exit}\,k_z). \tag{13}$$

7. Look up the next leaf: $\ell' =$ `amr_next_leaf`(i, f, x, y, z).

8. If $\ell' = 0$, the photon has escaped; set `photon%inside = .false.` and exit.

9. Apply the frequency shift (Section 6.2): $x_\nu \leftarrow x'_\nu$.

10. Set $\ell \leftarrow \ell'$ and continue.

6.2 Frequency Shift at Cell Boundaries

Each cell has its own Doppler frequency $\Delta\nu_{D,\ell}$ (because cells can have different temperatures) and its own bulk velocity component along the photon direction,

$$u_\ell = \tilde{v}_{x,\ell}\,k_x + \tilde{v}_{y,\ell}\,k_y + \tilde{v}_{z,\ell}\,k_z \quad \text{[dimensionless, in units of local } v_{\rm th}\text{]}. \tag{14}$$

When crossing from cell ℓ (old) to cell ℓ' (new), the photon frequency must be transformed to the comoving frame of the new cell. The physical frequency is invariant, so:

$$\nu_{\rm phys} = \nu_0 \left(1 + \frac{(x_\nu^{\rm old} + u_\ell)\,\Delta\nu_{D,\ell}}{\nu_0}\right), \tag{15}$$

and the new dimensionless frequency is:

$$\boxed{x_\nu^{\rm new} = (x_\nu^{\rm old} + u_\ell)\,\frac{\Delta\nu_{D,\ell}}{\Delta\nu_{D,\ell'}} - u_{\ell'}\,.} \tag{16}$$

This formula transforms between the comoving frames of two cells with different temperatures and bulk velocities in a single step.

6.3 Overshoot Correction

When $\tau > \tau_{\rm in}$ after stepping across a cell, the photon actually stopped inside the cell. The exact interaction distance is found by backtracking:

$$\Delta s_{\rm back} = \frac{\tau - \tau_{\rm in}}{\kappa}. \tag{17}$$

The total displacement from the entry point is then $d = d + t_{\rm exit} - \Delta s_{\rm back}$, and the final position is

$$(x, y, z) \leftarrow (\texttt{photon\%}x + d\,k_x,\ \texttt{photon\%}y + d\,k_y,\ \texttt{photon\%}z + d\,k_z). \tag{18}$$

Note that the position update is accumulated as a single floating-point addition from the *original* entry point (not from successive face crossings), which preserves numerical precision.

6.4 Photon Escape and Spectrum Accumulation

When the photon leaves the box ($\ell' = 0$), the frequency is converted to the lab frame:

$$x_\nu^{\rm lab} = x_\nu + u_\ell, \tag{19}$$

then scaled to the reference Doppler frequency:

$$x_\nu^{\rm ref} = x_\nu^{\rm lab}\,\frac{\Delta\nu_{D,\ell}}{\Delta\nu_{D,\rm ref}}. \tag{20}$$

The frequency bin index is

$$i_\nu = \left\lfloor \frac{x_\nu^{\rm ref} - x_{\rm min}}{\delta x} \right\rfloor + 1, \tag{21}$$

and the photon weight is accumulated into $\texttt{amr_grid\%Jout}(i_\nu)$.

6.5 Boundary Handling: Periodicity and Symmetry Planes

When `amr_next_leaf` returns no neighbor at a face crossing (`il_new` ≤ 0), the AMR raytrace inspects the same global flags that select the Cartesian `raytrace_to_tau_car_xyper`, `_xysym`, and `_xyzsym` variants:

`par%xy_periodic` (e.g. `plane_atmosphere`): the photon position wraps to the opposite face on the x and y sides,

$$\begin{aligned}
\text{face } +x: &\quad x \leftarrow x - \texttt{amr\%xrange},\\
\text{face } -x: &\quad x \leftarrow x + \texttt{amr\%xrange},\\
\text{face } +y: &\quad y \leftarrow y - \texttt{amr\%yrange},\\
\text{face } -y: &\quad y \leftarrow y + \texttt{amr\%yrange},
\end{aligned}$$

and the leaf is re-located via `amr_find_leaf`. Faces $\pm z$ remain escape boundaries.

`par%xy_symmetry` (mirror at $x = x_{\rm min}$ and $y = y_{\rm min}$): on the $-x$ face the direction is reflected ($k_x \to -k_x$); on the $-y$ face $k_y \to -k_y$. In both cases the photon stays in the same leaf (`il_new = il`). Other faces remain escape boundaries.

`par%xyz_symmetry` same as `xy_symmetry` with an additional $k_z \to -k_z$ on the $-z$ face.

Reflection assumption. The reflection is treated as specular at the leaf face, which assumes that no leaf cell straddles the symmetry plane. This holds for any generic AMR dataset whose octree root is built on the centered range $[-L/2, +L/2]$ and refined uniformly downward (the convention used by both the RAMSES and generic readers).

Frequency shift across reflection. For a cross-cell crossing the comoving-frame shift retains its usual form,

$$x'_\nu = (x_\nu + u_1)\,\frac{\Delta\nu_{D,\ell}}{\Delta\nu_{D,\ell'}} - u_2,$$

where u_1 uses the pre-crossing direction and u_2 uses the post-crossing direction. When the photon reflects within the same leaf, $\Delta\nu_{D,\ell}/\Delta\nu_{D,\ell'} = 1$, so

$$x'_\nu = (x_\nu + u_1) - u_2,$$

where u_2 is recomputed with the reflected $\hat{\mathbf{k}}$. This is equivalent to keeping the lab-frame frequency $x_\nu^{\rm lab} = x_\nu + u_1$ invariant under the direction flip, as required for a passive mirror plane.

Final position. `raytrace_to_tau_amr` updates the running (x, y, z) in place at every face crossing and at the in-cell overshoot exit, so the photon's final position remains correct after wraps and reflections without an external offset accumulator.

6.6 Other Raytrace Routines

Six additional routines in `raytrace_amr_mod` share the same AMR traversal kernel but accumulate different quantities:

Routine	Purpose
`raytrace_to_edge_amr`	Total τ to box edge (photon unchanged)
`raytrace_to_edge_tau_gas_amr`	Gas-only τ to box edge
`raytrace_to_edge_column_amr`	$N_{\rm HI}$ and dust τ to box edge
`raytrace_to_dist_amr`	Total τ to fixed distance s
`raytrace_to_dist_tau_gas_amr`	Gas-only τ to fixed distance s
`raytrace_to_dist_column_amr`	$N_{\rm HI}$ and dust τ to fixed distance s

All six routines propagate a copy of the photon state (photon is not modified) and apply the same frequency-shift formula at each cell crossing so that the Voigt opacity is evaluated at the correct comoving frequency in every cell. They share the same boundary handling described in Section 6.5: local copies of k_x, k_y, k_z are flipped on reflection so that `photon0` itself is never modified.

7. Scattering

The scattering routines in `scattering_amr.f90` mirror the Cartesian versions but read physical parameters from `amr_grid` arrays indexed by the leaf index `photon%icell_amr`.

7.1 Resonance Scattering

`scatter_resonance_nostokes_amr` implements the resonance scattering algorithm (no Stokes polarization). The scattering is handled by the procedure pointer `do_resonance`, which is set at initialization to one of `do_resonance1_amr`, `do_resonance2_amr`, etc., depending on the line type.

1. **Sample the scattering atom velocity.** The thermal velocity component of the atom along the photon direction (in the cell comoving frame) is sampled from the redistribution function via `rand_resonance_vz`:

$$u_z = x_\nu + u_\ell. \tag{22}$$

   The two perpendicular thermal velocity components (u_x, u_y) are drawn independently from a Gaussian with unit variance.

2. **Sample the scattering angle.** The polar scattering angle θ (between the incoming and outgoing photon directions) is drawn from the phase function

$$P(\cos\theta) = \frac{3}{8}E_1\cos^2\theta + \frac{4-E_1}{8}, \tag{23}$$

   via the function `rand_resonance`(E_1). The parameter E_1 is a property of the atomic transition:

   - $E_1 = 1$: pure Rayleigh scattering ($J = 0 \to J = 1 \to J = 0$, e.g. He I).
   - $0 < E_1 < 1$: mixed Rayleigh + isotropic (e.g. Lyα: $E_1 \approx 0$ for the $J = 1/2$ branch, $E_1 = 1$ for the $J = 3/2$ branch, giving an effective E_1 that depends on the occupation of upper sub-levels).
   - $E_1 = 0$: isotropic scattering.
   - $-1/2 \le E_1 < 0$: oblate phase function (forward and backward directions suppressed).

   The azimuthal angle ϕ is drawn uniformly from $[0, 2\pi)$.

3. **Compute the new photon frequency.** In the atom frame the frequency after scattering is

$$x'_\nu = x_\nu^{\rm atom} + u_z\cos\theta + (u_x\cos\phi + u_y\sin\phi)\sin\theta, \tag{24}$$

   where $x_\nu^{\rm atom} = x_\nu - u_z$ is the frequency in the atom rest frame. An optional recoil correction $\Delta x = -g_{\rm recoil}(1-\cos\theta)$ is applied if `par%recoil = .true.`

4. **Apply core-skip acceleration** if $|x_\nu| < x_{\rm crit}$.

The core-skip threshold $x_{\rm crit}$ is computed cell-by-cell at every scattering event by `amr_xcrit_local(il, x, y, z, . . . )` following Smith et al. (2015, Eq. 35) and the RASCAS convention:

$$a\tau_{\rm cell} = a_{\rm V}(il)\,\texttt{rhokap}(il)\,\Delta\ell_{\rm face}, \qquad x_{\rm crit} = \begin{cases} (a\tau_{\rm cell})^{1/3}/5 & a\tau_{\rm cell} > 1 \\ 0 & \text{otherwise} \end{cases}$$

where $\Delta\ell_{\rm face}$ is the minimum distance from the current photon position to any of the six leaf-cell faces, computed from the cell center (c_x, c_y, c_z) and half-width h as $\min(h-|x-c_x|, h-|y-c_y|, h-|z-c_z|)$.

The old behavior — a single volume-averaged $\langle x_{\rm crit}\rangle$ from $\langle a_{\rm V}\rangle$ `tauhomo` stored in `amr_grid%xcrit` — is retained behind `par%core_skip_global = .true.` for benchmarking only. Inhomogeneous boxes such as the jellyfish galaxy require the cell-by-cell formula: $\langle a\tau\rangle$ is small there because most of the box is hot, sparse gas, so the volume-averaged $x_{\rm crit}$ collapses to zero and the boost never engages even though individual galaxy-core cells have $a\tau_{\rm cell} \gtrsim 10^9$.

7.2 Dust Scattering

`scatter_dust_nostokes_amr` uses the standard Henyey–Greenstein phase function. When dust is present, the probability of the interaction being a dust event is

$$p_d = \frac{\bar{\kappa}_{d,\ell}}{\bar{\kappa}_{0,\ell}\,\phi(x_\nu, a_\ell) + \bar{\kappa}_{d,\ell}}, \tag{25}$$

sampled at the scattering location (cell ℓ).

7.3 Molecular-Hydrogen (H_2) Absorption on the AMR Grid

The molecular-hydrogen effect on Lyα (H_2 Lyman/Werner-band pumping and destruction; see the user manual, § "Molecular-Hydrogen Effect on Lyα") is fully supported on the AMR grid. `raytrace_to_tau_amr` adds the H_2 line opacity $\bar{\kappa}_{0,\ell}\,\kappa_{\rm H2}(x_\nu; \Delta\nu_{D,\ell})$ to the transport optical depth in each leaf, evaluated from the grid-agnostic core `h2_kappa_D(xfreq, Dfreq)` with the leaf Doppler width $\Delta\nu_{D,\ell} =$ `amr_grid%Dfreq`(ℓ). The scattering dispatch adds a three-way split among {H I resonance, dust, H_2}, and an H_2 event is handled by `scatter_h2` (which branches on `par%use_amr_grid` for the leaf Doppler width). The peel-off sight-line integral (`raytrace_to_edge_amr`) is likewise attenuated by H_2, and the absorbed-fraction budget is MPI-reduced and written in the AMR output path. The AMR and Cartesian absorbed fractions agree to $\sim$ 1–2% (octree volume averaging).

7.4 Multi-Level Resonance Transitions

For complex atoms (HeI, FeII) represented by multiple transitions, the AMR scattering module provides `do_resonance4_amr`, `do_resonance5_amr`, and `do_resonance6_amr`, which follow the same logic as their Cartesian counterparts but use cell-by-cell Doppler frequencies and Voigt parameters.

8. Output Normalization

After all photons are traced, the MPI reduction collects all frequency contributions via `MPI_ALLREDUCE` into `amr_grid%Jout` (`output_reduce_amr` in `output_sum_rect.f90`).

The normalized specific intensity (per unit frequency per steradian) is:

$$J_{\rm out}(x_\nu) = \frac{\texttt{Jout}(i_\nu)}{N_{\rm ph}\,\delta x\,2\pi\,A}, \tag{26}$$

where $N_{\rm ph}$ is the number of photons, δx the frequency bin width, and

$$A = 4\pi\, d_{\rm cm}^2 \tag{27}$$

is the surface area of a unit sphere in cm^2 (the factor $d_{\rm cm}$ is `par%distance2cm`).

For dimensionless runs (`distance2cm = 1`, no `distance_unit`), this reduces to $A = 4\pi$, making $J_{\rm out}$ independent of the arbitrary box size. This matches the Cartesian convention ($A = 4\pi\, r_{\rm max}^2\, d_{\rm cm}^2$) when $r_{\rm max} = L_{\rm box}/2$ and the same `distance2cm` is used.

9. Core-Skip Acceleration

Lyman-α photons scatter extremely frequently in the line core ($|x_\nu| \ll 1$), where the optical depth per cell is large. The core-skip algorithm (Laursen et al. 2009) exploits the fact that a photon in the core will scatter many times within a single cell without traveling far. When $|x_\nu| < x_{\rm crit}$, the frequency is redrawn from the wing distribution directly, skipping the core-scattering steps.

In AMR mode, $x_{\rm crit}$ is computed from the mean opacity product:

$$(a\tau_0)_{\rm cell} = \bar{a}\,\bar{\tau}_0 \;/\; N_{\rm cell,axis}, \tag{28}$$

where $N_{\rm cell,axis} = L_{\rm box}/(2\,h_{\rm root})$ is the number of root cells along one axis. The threshold is then

$$x_{\rm crit} = \begin{cases} 0.02\,\exp[\xi\,(\ln(a\tau_0)_{\rm cell})^{\chi}] & (a\tau_0)_{\rm cell} > 1, \\ 0 & \text{otherwise}, \end{cases} \tag{29}$$

with $(\xi, \chi) = (0.6, 1.2)$ for $(a\tau_0)_{\rm cell} \le 60$ and $(1.4, 0.6)$ otherwise.

10. HEALPix Inside-Observer Mode

When the user sets `par%nside` > 0, `setup.f90` automatically enables `par%observer_located_inside = .true.` and switches the entire output and peel-off chain to HEALPix all-sky maps. This mode is intended for "galactic" observations where the observer sits at a chosen position (o_x, o_y, o_z) *inside* the medium; each escaping photon contributes to the all-sky map indexed by the direction from the observer back to the photon, computed with `vec2pix(observer%nside, ...)`.

For the AMR path, a parallel set of `_inside_amr` routines mirrors the Cartesian `_inside` chain:

Routine	File
`peeling_direct_inside_amr`	`peelingoff_amr.f90`
`peeling_dust_nostokes_inside_amr`	`peelingoff_amr.f90`
`peeling_resonance_nostokes_inside_amr`	`peelingoff_amr.f90`
`output_reduce_inside_amr`	`output_sum_heal.f90`
`output_normalize_inside_amr`	`output_sum_heal.f90`
`make_sightline_tau_inside_amr`	`sightline_tau_heal.f90`

In v2.10 (class-based dispatch), six `amr_*` type-bound methods in `grid_class.f90` branch on `par%observer_located_inside`:

Method	inside path	outside path (default)
`amr_output_reduce`	`output_reduce_inside_amr`	`output_reduce_amr`
`amr_output_normalize`	`output_normalize_inside_amr`	`output_normalize_amr`
`amr_write_output`	`write_output_inside`	`write_output_outside`
`amr_observer_create`	`observer_create_inside`	`observer_create_outside`
`amr_observer_destroy`	`observer_destroy_inside`	`observer_destroy_outside`
`amr_make_sightline_tau`	`make_sightline_tau_inside_amr`	`make_sightline_tau_amr`

The peeling helpers receive `amr` (`type(amr_grid_type), intent(in)`) as an explicit parameter rather than reading the global `amr_grid`; this keeps the v2.10 OOP interface clean and removes the implicit module-global coupling. Each peel routine computes the direction from the photon to the observer,

$$\hat{k}_{\rm obs} = \frac{\mathbf{r}_{\rm obs} - \mathbf{r}_{\rm ph}}{|\mathbf{r}_{\rm obs} - \mathbf{r}_{\rm ph}|}, \tag{30}$$

applies the Doppler shift in the local cell using $\Delta\nu_D$ = `amr%Dfreq(il)` and $\tilde{v}$ = `amr%vfx,y,z(il)`, traces the optical depth from the photon position to the observer with `call raytrace_to_dist_amr(pobs, amr, r, tau)`, and deposits the peeled weight into either `observer(i)%direc_heal[_2D]` (direct) or `observer(i)%scatt_heal[_2D]` (scattered).

Peeling dispatch is performed inside the scatter / generate_photon modules:

Module	Action
`generate_photon_amr.f90`	if `par%save_peeloff`
	if `par%observer_located_inside` → `peeling_direct_inside_amr`
	else if `point_illumination/stellar_illumination` → corresponding _amr
	else → `peeling_direct_amr`
`scattering_amr.f90`	if `par%save_peeloff`
	if `par%observer_located_inside` →
	`peeling_{dust,resonance}_nostokes_inside_amr`
	else → `peeling_{dust,resonance}_nostokes_amr`

Source scope. The AMR `_inside` path supports the same internal sources as the Cartesian `_inside` path: `point`, `uniform`, `gaussian`, `exponential`, `sersic`, and `diffuse_emissivity`. External-illumination geometries (`point_illumination`, `stellar_illumination`, `plane_illumination`) are only meaningful for an observer outside the medium and remain restricted to the `_outside` path. Stokes polarization is not yet implemented for the HEALPix `_inside` path (mirroring the Cartesian convention).

Normalization. `output_normalize_inside_amr` uses the AMR convention $A = 4\pi\, d_{\rm cm}^2$ (independent of $L_{\rm box}$) to normalize `Jout`, `Jin` and `Jabs`, and $A_{\rm pix} = \Omega_{\rm pix}\, d_{\rm cm}^2$ for the HEALPix observer pixels, identical to the Cartesian `output_normalize_inside` modulo the area choice.

Sightline tau. `make_sightline_tau_inside_amr` walks from the observer position to each of the six box faces (±`x`, ±`y`, ±`z`) by stepping along the relevant axis until reaching the `amr%xmin/xmax/ymin/ymax/zmin/zmax` boundary, accumulating τ and N(HI) via `raytrace_to_dist_tau_gas_amr` and `raytrace_to_dist_column_amr`.

The outside-observer counterpart `make_sightline_tau_amr` (in `sightline_tau_amr.f90`) is structurally analogous: for each pixel of the (`nxim, nyim`) detector grid it places a virtual photon at the far face of the AMR box and integrates inward toward the observer using `raytrace_to_edge_tau_gas_amr` and `raytrace_to_edge_column_amr`.

$\tau_{\rm huge}$ early-exit (sight-line vs. photon transport). The AMR raytrace routines used by the peel-off (`raytrace_to_edge_amr`, `raytrace_to_edge_amr_dust_Ha`, `raytrace_to_dist_amr`) early-exit when the running optical depth exceeds $\tau_{\rm huge} = 300$ (`tau_huge` in `define.f90`, shared with the Cartesian and clump peel-off); beyond it $\exp(-\tau) < 5 \times 10^{-131}$ and accumulating further cannot change the peel-off weight. Until 2026-10-20 the

cap was 745.2, the double-precision underflow point of $\exp(-\tau)$; lowering it left the spectra unchanged and shortened a static clumpy run from 76.3 to 42.2 min (`Models_for_HGKym/docs/RTscat_comparison.pdf`). The sight-line variants `raytrace_to_edge_tau_gas_amr` and `raytrace_to_dist_tau_gas_amr` exist precisely to *report* the integrated τ, however large, and therefore run *without* that cap (matching the Cartesian `raytrace_to_edge_tau_gas` and `raytrace_to_dist_tau_gas_car` in `raytrace_car.f90`). A short-lived bug (fixed 2026-05-04) had the cap inadvertently applied in `raytrace_to_edge_tau_gas_amr`, capping outside-observer sight-line τ at ~ 745 and producing AMR/Cartesian disagreement of an order of magnitude on the line-center maps.

Cleanup convention (`grid_destroy_amr`). `grid_destroy_amr` now mirrors the Cartesian and clump `grid_destroy` routines: it issues a single `destroy_shared_mem_all()` (idempotent when `num_windows == 0`), nullifies every shared-memory pointer field of `amr_grid`, and Fortran-deallocates only the genuinely allocatable output arrays local to each rank (`Jout`/`Jin`/`Jabs`/`Jmu`). Calling `destroy_mem` on each shared-memory pointer individually (the old behavior) crashes whenever another module (`observer_destroy_outside`, `grid_destroy_clump`) has already freed the global window registry: the array-by-array routine falls through to a Fortran `deallocate()` of MPI-window memory and trips ifort `severe (173)`.

Smoke test (uniform sphere, point source at origin, $T = 10^4$ K, $\tau_0 = 10^4$, observer at $o_x = 0.3$, nside = 8, 10^6 photons).

Quantity	AMR	Cartesian (64^3)
$\sum$ Jout	8.5134×10^{-2}	8.5136×10^{-2}
$\sum$ Jin	8.5135×10^{-2}	8.5136×10^{-2}
$\sum$ Scattered HEALPix	2592	2322
$\sum$ Direct HEALPix	5.14×10^{-2}	4.66×10^{-2}

The `Jout`/`Jin` agreement matches the `_outside` validation, and the $\sim 12\%$ difference in $\sum$ Scattered is consistent with the documented octree volume-averaging of the sphere boundary plus shot noise from 10^6 photons. The Direct HEALPix sum is dominated by very few unscattered photons at $\tau_0 = 10^4$.

11. Validation

The AMR implementation was validated against equivalent Cartesian runs for a uniform-density sphere of HI gas with a central point source ($T = 10^4$ K, no dust).

The AMR sphere is described by the generic text format with the same set of leaf cells at the same AMR level, providing effectively the same resolution as the 101^3 Cartesian grid. The Cartesian comparison uses $r_{\rm max} = L_{\rm box}/2$ with no `distance_unit`.

τ_0	$J^{\rm AMR}_{\rm out,max}$	$J^{\rm CAR}_{\rm out,max}$	Ratio (AMR/CAR)	Peak velocity
10^2	6.630×10^{-3}	6.836×10^{-3}	0.970	$\pm 26.75\,{\rm km\,s^{-1}}$
10^4	7.144×10^{-3}	7.402×10^{-3}	0.965	$\pm 38.21\,{\rm km\,s^{-1}}$

Peak positions, column densities $N(\mathrm{HI})_{\mathrm{pole}}$, and τ_{pole} agree exactly between AMR and Cartesian runs. The $\sim$ 3–5% amplitude difference is entirely attributable to the octree voxelisation of the sphere boundary: leaf cells that straddle the sphere edge carry only the average density, softening the sharp boundary. No normalization error is present.

12. Summary of Key Design Choices

1. **Flat linear octree.** All cells (internal and leaf) are stored in a single 1-indexed Fortran array. Parent–child relationships are maintained through integer index arrays. This avoids pointer indirection and is cache-friendly.

2. **Precomputed face-neighbor table.** Built once before the simulation loop. Boundary crossings cost O(1) (plus O($\Delta\ell$) descent into finer cells if the neighbor is more refined).

3. **Cell-by-cell Voigt parameters.** Each leaf stores its own $\Delta\nu_D$ and a, supporting arbitrary temperature distributions without any approximation.

4. **Frequency shift at every cell crossing.** The comoving-frame frequency is updated at each boundary, so the Voigt opacity is always evaluated in the correct local frame, including bulk velocity effects.

5. **Single floating-point accumulation of displacement.** The photon position is updated as $\mathbf{r}_0 + d\,\hat{k}$ rather than as a sequence of increments, preserving numerical precision across many cell crossings.

6. **OOP class hierarchy for AMR/Cartesian switching.** A Fortran 2003 abstract base type `grid_base_type` (in `grid_class.f90`) declares all physics operations as deferred type-bound procedures. The concrete types `grid_car_type` and `grid_amr_type` implement them by delegating to the existing `_car` and `_amr` routines. The polymorphic variable `class(grid_base_type), allocatable :: grid` in `main.f90` is allocated to the appropriate type at runtime based on `par%grid_type`. `setup.f90` no longer assigns any global procedure pointers; all dispatch goes through `grid%method()` calls.

7. **Pole-traversal tau normalization.** The AMR raytrace kernel itself is used to compute τ_{pole}, giving an exact normalization that is consistent with the Cartesian convention and avoids volume-average errors.